SoSe 26 Type Theory & Logic & Functional Programming Notes

Igor Dimitrov

2026-04-21

Table of contents

Preface

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

Part I

Logic

1 Lecture 1 - Propositional Logic: Syntax

1.1 Overview

Brief summary of what this lecture does (2–4 lines).

Goal: Define the expression we can write, these are called **well-formed-formulas**, or **wff**s.

There are various ways of defining what counts as a wff. One of them is the inductive approach.

1.2 Inductive Approach

First we start with **atomic formulas**:

$$P_0, P_1, P_2, ...$$
$$(P_i)_{i \in \mathbb{N}}$$

are atomic propositions.

We give some rules to inductively define new formulas from old ones.

1) if φ, ψ are **wff** then $\varphi \Rightarrow \psi$ is a **wff**
2) if φ, ψ are **wff** then $\varphi \wedge \psi$ is a **wff**
3) if φ, ψ are **wff** then $\varphi \vee \psi$ is a **wff**
4) if φ is a **wff** then $\neg\varphi$ is a **wff**

1.3 Constructor Approach

It is useful to express these rules with **constructors**, which define wffs as values of some functions (with codomain WFF)

In this approach atomic formulas are values of functions:

Definition 1.1 (Atomic formulas)**.** Atomic formulas are values of functions like:

$$P : \mathbb{N} \to \text{WFF}, \quad : i \mapsto P_i$$

or

$$''\cdot'' : \text{String} \to \text{WFF}, \quad : x \mapsto "x"$$

Example 1.1 (String Constructor)**.** with the latter we can construct arbitrary propositions like:

"today␣is␣sunny"

We also have constructors for the logical connectives:

Definition 1.2 (Constructors for Logical Connectives)**.**

$$\begin{aligned}
&(\neg) : \text{WFF} \to \text{WFF}, \quad : \varphi \mapsto \neg\varphi \\
&(\wedge) : \text{WFF} \times \text{WFF} \to \text{WFF}, \quad : (\varphi, \psi) \mapsto \varphi \wedge \psi \\
&(\vee) : \text{WFF} \times \text{WFF} \to \text{WFF}, \quad : (\varphi, \psi) \mapsto \varphi \vee \psi \\
&(\Rightarrow) : \text{WFF} \times \text{WFF} \to \text{WFF}, \quad : (\varphi, \psi) \mapsto \varphi \Rightarrow \psi
\end{aligned}$$

Remark 1.1 (Constructor vs Symbol View of Logical Connectives). Earlier we wrote $\varphi \Rightarrow \psi$ and then we denoted $\Rightarrow$ symbol as a function of type $\text{WFF} \times \text{WFF} \to \text{WFF}$, we use $(\Rightarrow)$ to distinguish between the two concepts.

A different but related question that arises si the ambiguity of $P_1 \Rightarrow P_2 \Rightarrow P_3$. According to the inductive definition there are two ways this formula could have been constructed which corresponds to the different formulas:

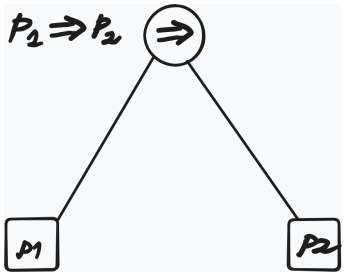
1. $(P_1 \Rightarrow P_2) \Rightarrow P_3$
2. $P_1 \Rightarrow (P_2 \Rightarrow P_3)$

To resolve this ambiguity we define the associativity rule:

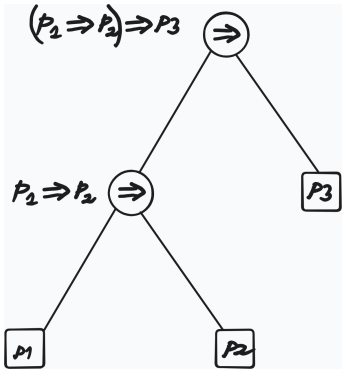
Definition 1.3 (Associativity Rule for $\Rightarrow$)**.**

$$P_1 \Rightarrow P_2 \Rightarrow P_3 := (P_1 \Rightarrow P_2) \Rightarrow P_3$$

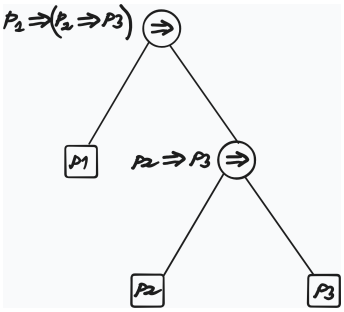
Remark 1.2 (Wffs as Trees). Wffs can be viewed as trees. Trees encode the derivation structure and resolve the ambiguity



(a) tree 1



(b) tree 2



(c) tree 3

our collection of wffs can be further enriched with another binary constructor:

$$\begin{aligned}(\Leftrightarrow) &: \mathrm{WFF} \times \mathrm{WFF} \to \mathrm{WFF} \\ &: (\varphi, \psi) \mapsto \varphi \Leftrightarrow \psi\end{aligned}$$

Remark 1.3 (Meaning of $\Leftrightarrow$). At this stage there is no relation between

$$\varphi \Leftrightarrow \psi \quad \text{and} \quad (\varphi \Rightarrow \psi) \wedge (\psi \Rightarrow \varphi)$$

Let is give another 'set-theoretic' approach

1.4 Set Theoretic Approach

First we define the alphabets:

Definition 1.4 (Set Theoeretic Approach to Defining Wffs)**.** We start with the alphabets:

$$\begin{array}{ll}\mathcal{A} := \{P_i\}_{i\in\mathbb{N}} & \text{(sub-alphabet of atomic formulas)} \\ \mathcal{C} := \{\Rightarrow, \wedge, \vee, \neg, \Leftrightarrow\} & \text{(sub-alphabet of logical connectives)} \\ \mathcal{V} := \mathcal{A} \cup \mathcal{C} \cup \{(,)\} & \text{(alphabet of propositional logic, including punctuation marks)}\end{array}$$

Then we define the so-called **Kleene Closure** of $\mathcal{V}$:

Definition 1.5 (Kleene Closure of $\mathcal{V}$)**.**

$$\mathcal{V}^* := \bigcup_{i=0}^{\infty} \mathcal{V}_i$$

$\mathcal{V}^*$ is the language containing all sorts of strings that can be constructed from the alphabet $\mathcal{V}$, also senseless ones.

To define the language propositional logic that contains only the formulas that we want and nothing else, we take the usual set-theoretic approach of taking the infinite union of all sets $\mathcal{W} \subseteq \mathcal{V}$ that satisfy the following properties:

1) $\mathcal{A} \subseteq \mathcal{W}$
2) if $\varphi, \psi \in \mathcal{W}$ then $\varphi \diamond \psi \in \mathcal{W}$, where $\diamond \in \{\Rightarrow, \wedge, \vee, \Leftrightarrow\}$
3) if $\varphi \in \mathcal{W}$ then $\neg\varphi \in \mathcal{W}$

Then the language of propositinal logic is defined as:

Definition 1.6 (Language Propositional Logic as Infinite Intersection)**.**

$$\mathcal{F}(\mathcal{A}) := \bigcap \{\mathcal{W} \subseteq \mathcal{V}^* | \mathcal{W} \text{ satisfies (1), (2), and (3)}\}$$

Theorem 1.1.

With this construction $\mathcal{F}(\mathcal{A})$ is the smallest set that satisfies the properties (1), (2) and (3), i.e.

1) $\mathcal{F}(\mathcal{A})$ satisfies the properties (1), (2) and (3)
2) For any set $\mathcal{W}$ that satisfies the properties (1), (2) and (3) we have $\mathcal{W} \subseteq \mathcal{F}(\mathcal{A})$

1.5 Summary

- WFF is defined inductively

- Constructors that generate WFFs
- Set theoretic construction of WFFs

2 Lecture 2 - Height and Subformulas

2.1 Overview

- We provide a fourth, iterative set construction for the language of propositional logic.
- We give two equivalent definitions for the height of a formula.
- We give two equivalent definitions of a function that gives the list of subformulas of a formula

2.2 Recurisve Set Construction

We define $\mathcal{F}_0$ iteratively as follows:

$$\begin{aligned}
&\mathcal{F}_0 := \mathcal{A} \\
&\mathcal{F}_1 := \mathcal{F}_0 \cup \{\, \neg\varphi \mid \varphi \in \mathcal{F}_0 \,\} \cup \{\, \varphi \diamond \psi \mid \varphi, \psi \in \mathcal{F}_0 \,\} && (\diamond \in \mathcal{C}) \\
&\mathcal{F}_n := \mathcal{F}_{n-1} \cup \{\, \neg\varphi \mid \varphi \in \mathcal{F}_{n-1} \setminus \mathcal{F}_{n-2} \,\} \cup \{\, \varphi \diamond \psi \mid \varphi, \psi \in \mathcal{F}_{n-1} \,\} && (n \geq 2)
\end{aligned}$$

With this construction it follows that:

$$\mathcal{F}_n \subseteq \mathcal{F}_{n+1}$$

Example 2.1. $\neg\neg P_4 \in \mathcal{F}_2 \subseteq \mathcal{F}_3 \subseteq ...$

but $\neg\neg P_4 \notin \mathcal{F}_1$, because $\neg P_4 \notin \mathcal{F}_0$

Now we make the following definition:

Definition 2.1.

$$\mathcal{F} := \bigcup_{n \in \mathbb{N}} \mathcal{F}_n$$

and we claim:

Lemma 2.1.

$$\mathcal{F} = \mathcal{F}(\mathcal{A})$$

Where $\mathcal{F}(\mathcal{A})$ is defined in Definition 1.6

Proof. To prove $\mathcal{F} \subseteq \mathcal{F}(\mathcal{A})$, it suffices to show that

$$\mathcal{F}_n \subseteq \mathcal{F}(\mathcal{A}), \quad \forall n, \text{ by induction on } n$$

To conclude the proof we need to show that $\mathcal{F}_A \subseteq \mathcal{F}$ □

Definition 2.2 (Height of Formula - Set Approach)**.** For all $\varphi \in \mathcal{F}$, there is a well defined natural number $\mathrm{ht}(\varphi) \in \mathbb{N}$:

$$\mathrm{ht}(\varphi) := \min\{\, n \in \mathbb{N} \mid \varphi \in \mathcal{F}_n \,\}$$

called the **height** of φ

We give another, a more computational definition for the height.

Definition 2.3 (Height of a Formula - Computational Approach)**.** We define $\mathrm{ht}(\cdot)$ recursively on the structure of a formula ϕ as follows:

$$\begin{aligned}
&\mathrm{ht}(P_i) = 0 && (P_i \in \mathcal{A}) \\
&\mathrm{ht}(\neg\varphi) = 1 + \mathrm{ht}(\varphi) \\
&\mathrm{ht}(\varphi \diamond \psi) = 1 + \max(\mathrm{ht}(\varphi), \mathrm{ht}(\psi)) && (\diamond \in \mathcal{C})
\end{aligned}$$

Remark 2.1 (Pattern Matching). Such inductive (recursive) definitions are also called **pattern matching**

Example 2.2 (Derivation of Height)**.**

$$\begin{aligned}
\mathrm{ht}(\neg\neg(P_1 \Rightarrow P_2)) &= 1 + \mathrm{ht}(\neg(P_1 \Rightarrow P_2)) \\
&= 1 + 1 + \mathrm{ht}(P_1 \Rightarrow P_2) \\
&= 1 + 1 + \max(\mathrm{ht}(P_1), \mathrm{ht}(P_2)) \\
&= 1 + 1 + \max(1, 1) \\
&= 1 + 1 + 1 \\
&= 3
\end{aligned}$$

Obervation:

We observe that the height of a formula is the height of its syntax tree

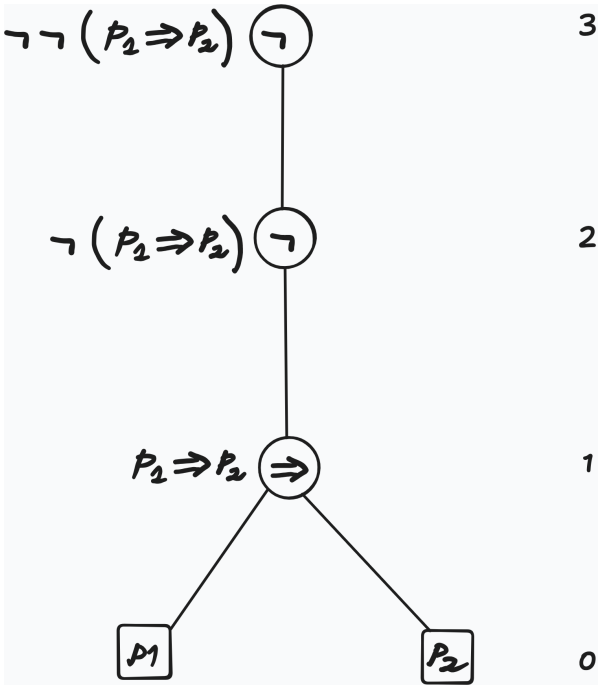


Figure 2.1: height of the syntax tree

We want to define a function

$$\mathrm{sf} : \mathcal{F}(\mathcal{A}) \to \mathcal{P}(\mathcal{F}(\mathcal{A}))$$

mapping a formula to its set of subformulas. For this we are going to define functions $\mathrm{sf}_i : \mathcal{F}_i \to \mathcal{P}(\mathcal{F})$ as follows:

Definition 2.4 (Set Construction of sf)**.**

$$\begin{aligned}
&\mathrm{sf}_0(P_i) := \{P_i\} \\
&\mathrm{sf}_n(\neg\varphi) := \{\neg\varphi\} \cup \mathrm{sf}_{n-1}(\varphi) \\
&\mathrm{sf}_n(\varphi \diamond \psi) := \{\varphi \diamond \psi\} \cup \mathrm{sf}_{n-1}(\varphi) \cup \mathrm{sf}_{n-1}(\psi)
\end{aligned}$$

Then sf can be defined as

$$\begin{aligned}
\mathrm{sf} &: \mathcal{F} \to \mathcal{P}(\mathcal{F}) \\
&: \varphi \mapsto \mathrm{sf}_{\mathrm{ht}(\varphi)}(\varphi)
\end{aligned}$$

We give another, more computational approach, by defining sf recursively on the structure of wffs as a function $\mathrm{sf} : \mathrm{WFF} \to [\mathrm{WFF}]$

Definition 2.5 (List Construction of sf)**.**

$$\begin{aligned}
&\mathrm{sf}(P_i) := [P_i] \\
&\mathrm{sf}(\neg\varphi) := [\neg\varphi] +\!\!+ \mathrm{sf}(\varphi) \\
&\mathrm{sf}(\varphi \diamond \psi) := [\varphi \diamond \psi] +\!\!+ \mathrm{sf}(\varphi) +\!\!+ \mathrm{sf}(\psi)
\end{aligned}$$

Example 2.3 (Derivation of the List of Subformulas with sf)**.**

$$\begin{aligned}
\mathrm{sf}(P_1 \vee \neg P_2) &= [P_1 \vee \neg P_2] +\!\!+ \mathrm{sf}(P_1) +\!\!+ \mathrm{sf}(\neg P_2) \\
&= [P_1 \vee \neg P_2] +\!\!+ [P_1] +\!\!+ [\neg P_2] +\!\!+ \mathrm{sf}(P_2) \\
&= [P_1 \vee \neg P_2] +\!\!+ [P_1] +\!\!+ [\neg P_2] +\!\!+ [P_2] \\
&= [P_1 \vee \neg P_2, P_1, \neg P_2, P_2]
\end{aligned}$$

2.3 Summary

- iterative set construction $\mathcal{F}_i$ - the language of propositional logic as infinite union of such
- two definitions for ht - height of a formula
- two definitions for sf - list or set of subformulas of a formula

3 Boolean Values and Pattern Matching

Logic and Functional Programming
Florent Schaffhauser
Heidelberg University
(Summer 2026)

This note introduces Boolean values in Rocq, basic pattern matching, curried functions, and the first small proofs about Boolean functions. The original lecture file was a Rocq source file; here the surrounding comments have been turned into prose, and Rocq commands are placed in executable Quarto code blocks.

3.1 Booleans in Rocq's Core Library

Boolean values are available without any imports in Rocq. There are two canonical Boolean values:

- `true`
- `false`

Both are values of type `bool`.

```
Check true.
Check false.
```

```
true
     : bool

false
     : bool
```

In Rocq, every expression has a type. The expressions `true` and `false` have type `bool`. The type `bool` itself also has a type: Rocq classifies it as `Set`.

Roughly speaking, `Set` is the universe of computational data types, such as Booleans, natural numbers, lists, and other datatypes. More generally, `Set` itself lives in a larger universe called `Type`.

```
Check bool.
About bool.
Check Set.
Check Type.
```

```
bool
     : Set

bool : Set

bool is not universe polymorphic
Expands to: Inductive Corelib.Init.Datatypes.bool
Declared in library Corelib.Init.Datatypes, line 39, characters 10-14

Set
     : Type

Type
     : Type
```

A useful first picture is:

```
true  : bool
false : bool
bool  : Set
Set   : Type
```

So the hierarchy starts with ordinary values, then their types, and then universes classifying those types.

> **Note**
>
> In type theory, types are themselves objects that can be checked and classified. This is why commands like `Check bool`, `Check Set`, and `Check Type` are meaningful.

Certain pre-defined functions are also available out of the box and can be used to compute expressions. For example, `negb` is Boolean negation.

```
Check negb.
About negb.
```

```
Check negb false.
Compute negb false.
```

```
negb
     : bool -> bool

negb : bool -> bool

negb is not universe polymorphic
Arguments negb b%bool_scope
negb is transparent
Expands to: Constant Corelib.Init.Datatypes.negb
Declared in library Corelib.Init.Datatypes, line 84, characters 11-15

negb false
     : bool

     = true
     : bool
```

Curried Notation

A function of two variables like `orb` is written in curried notation. This means that the value of `orb` on two Boolean arguments is written as

```
orb true false
```

not as

```
orb (true, false)
```

The second expression would suggest that `orb` takes a pair as input. But Rocq's `orb` takes one Boolean argument first, and then returns a new function waiting for the second Boolean argument.

```
Check orb true false.
Check orb.
Check bool -> (bool -> bool).
Check orb true.
```

```
(true || false)%bool
     : bool

orb
     : bool -> bool -> bool

bool -> bool -> bool
     : Set

orb true
     : bool -> bool
```

The type

```
bool -> bool -> bool
```

should be read as

```
bool -> (bool -> bool)
```

because arrow types associate to the right. Therefore, `orb true` is a function of type `bool -> bool`.

> **Note**
>
> A function type `A -> B -> C` means `A -> (B -> C)`. A function of two arguments is therefore a function that takes one argument and returns another function.

We can compute using this syntax.

```
Compute negb (orb true false).
```

```
     = false
     : bool
```

In the rest of the file, we construct basic functions from `bool` to `bool` and from `bool` to `bool -> bool`. Most of them are already contained in Rocq's core library, but redefining them is a good way to learn pattern matching.

3.2 Basic Pattern Matching

To avoid conflicts with Rocq's core library, we wrap everything in a module called `BooleanValues`. This is optional here, since we do not intend to use the current file as a library later on.

```
Module BooleanValues.
```

Inside the module, our own definitions can have the same names as functions from Rocq's core library. For example, when we define our own `negb`, it shadows the library function of the same name inside this module. The original library function still exists, but the unqualified name `negb` will refer to our local definition.

> **Note**
>
> A module gives us a local namespace. This lets us experiment with names like `negb`, `andb`, and `orb` without permanently overwriting Rocq's library definitions.

Boolean Negation

Let us start with the definition of the `negb` function, which sends `true` to `false` and `false` to `true`. We define it by pattern matching on Boolean values.

```
Definition negb (b : bool) : bool :=
  match b with
  | true  => false
  | false => true
  end.

Check negb.
```

```
negb
     : bool -> bool
```

The definition says: to compute `negb b`, inspect the possible forms of `b`.

- If `b` is `true`, return `false`.
- If `b` is `false`, return `true`.

Since `bool` has exactly two constructors, `true` and `false`, these two cases are exhaustive.

```
About negb.
Compute negb false.
```

```
negb : bool -> bool

negb is not universe polymorphic
Arguments negb b%bool_scope
negb is transparent
Expands to: Constant Top.BooleanValues.negb
Declared in toplevel input, characters 11-15

     = true
     : bool
```

We can also give expressions a name and then use that name in later computations.

```
Compute negb (negb false).

Definition example_bool := negb false.

Compute negb example_bool.
```

```
     = false
     : bool

     = false
     : bool
```

> **Note**
>
> Pattern matching is Rocq's way of defining functions by cases. For `bool`, there are exactly two cases: `true` and `false`.

Boolean Conjunction

Now we define an `andb` function. It takes two Booleans `b` and `b'`, and returns their Boolean conjunction.

The truth table for Boolean conjunction is:

b	b'	andb b b'
true	true	true
true	false	false
false	true	false
false	false	false

This truth table can be translated directly into a pattern match with four cases.

```
Definition andb (b b' : bool) : bool :=
  match b, b' with
  | true,  true  => true
  | true,  false => false
  | false, true  => false
  | false, false => false
  end.
```

In fact, some simplifications are possible. If the first Boolean is `true`, then the result is just the second Boolean. If the first Boolean is `false`, then the result is always `false`, regardless of the second Boolean.

```
match b, b' with
| true,  b' => b'
| false, _  => false
end
```

The symbol `_` is a wildcard pattern. It means: there is some value here, but we do not need to name it because the result does not depend on it.

Another possible, but less readable, way is to pattern match first on `b` and then on `b'`. This produces a nested program. It typechecks, but it is more verbose. Note that only the final `end` is followed by a period.

```
match b with
| true =>
  match b' with
  | true  => true
  | false => false
```

```
  end
| false =>
  match b' with
  | true  => false
  | false => false
  end
end.
```

As with `negb`, we can use `andb` to compute Boolean values.

```
Compute andb true false.
Compute andb true (negb example_bool).
```

```
     = false
     : bool

     = false
     : bool
```

Recall that `andb` is a function of two variables, but it is written in curried notation. If we ask Rocq to check its type, we get `bool -> bool -> bool`, not a function type from pairs.

```
Check andb.
Check andb true.
Compute andb true.
```

```
andb
     : bool -> bool -> bool

andb true
     : bool -> bool

     = fun b' : bool => if b' then true else false
     : bool -> bool
```

The expression `andb true` is a partially applied function. It is the function that takes a Boolean `b'` and returns `andb true b'`, which is just `b'`.

We see in particular that arrow types associate to the right:

```
bool -> bool -> bool
=
bool -> (bool -> bool)
```

This means that an expression like

```
andb b b'
```

is parsed as

```
(andb b) b'
```

In contrast, the arrow type `(bool -> bool) -> bool` describes a function which takes a Boolean function as input and returns a Boolean value. We can define an example by evaluating a function `f : bool -> bool` at a given Boolean value.

```
Definition eval_at_true : (bool -> bool) -> bool :=
  fun f => f true.

Compute eval_at_true negb.
Compute eval_at_true (andb true).

Definition eval_at (b : bool) : (bool -> bool) -> bool :=
  fun f => f b.

Compute eval_at false negb.
Compute eval_at false (andb true).
```

```
     = false
     : bool

     = true
     : bool

     = true
     : bool

     = false
     : bool
```

Boolean Disjunction

Let us define an `orb` function on Booleans.

The truth table for Boolean disjunction is:

b	b'	orb b b'
true	true	true
true	false	true
false	true	true
false	false	false

If the first Boolean is `true`, then the result is always `true`. If the first Boolean is `false`, then the result is the second Boolean.

```
Definition orb (b b' : bool) : bool :=
  match b, b' with
  | true,  _  => true
  | false, b' => b'
  end.
```

Since the first branch does not actually use `b'`, and the second branch only returns `b'`, we can also write the same idea as a match on `b` alone.

```
match b with
| true  => true
| false => b'
end
```

```
Compute orb false false.
Compute orb false (negb false).
```

```
     = false
     : bool

     = true
     : bool
```

Boolean Implication

Let us define Boolean implication. The intended truth table is:

b	b'	implb b b'
true	true	true
true	false	false
false	true	true
false	false	true

This corresponds to the classical idea that `P -> Q` is false only when `P` is true and `Q` is false.

For this one, we use Rocq's `if ... then ... else ...` syntax.

```
Definition implb (b b' : bool) : bool :=
  if b then b' else true.
```

Here

```
if b then b' else true
```

is essentially shorthand for the following pattern match:

```
match b with
| true  => b'
| false => true
end
```

So `if ... then ... else ...` is not a separate mysterious mechanism. For Booleans, it is a convenient notation for pattern matching on a Boolean condition.

Exercise: implement the function `implb` using pattern matching. The result of the following computation should be the Boolean `true`.

```
Compute implb false true.
```

```
     = true
     : bool
```

Exercise: implement the Boolean if-then-else function as a function with type signature

```
bool -> bool -> bool -> bool
```

It should take three Boolean arguments:

```
condition -> then_branch -> else_branch -> result
```

One possible implementation is:

```
Definition ifb (b x y : bool) : bool :=
  match b with
  | true  => x
  | false => y
  end.
```

3.3 Theorem Proving

Next, we start using Rocq as a tool for proving properties of the functions we have defined. The syntax looks different at first, but in type theory stating a theorem is closely related to declaring a type, and proving a theorem is closely related to constructing a term of that type.

For example, the statement

```
negb (negb true) = true
```

is an equality proposition. To prove it means to construct a proof object inhabiting that proposition.

> **Note**
>
> In Rocq, a theorem statement is a type. Proving the theorem means constructing a term whose type is the theorem statement.

Negation Is Involutive

The first property that we prove is that, for every Boolean value `b`, we have

```
negb (negb b) = b
```

In ordinary language: applying Boolean negation twice gives us back the original Boolean value.

Let us start with some special cases, in which we prove our equality by direct computation. When using an interactive Rocq editor, the tactic state shows the current goal and helps guide the proof.

```
Theorem negb_negb_true_eq_true : negb (negb true) = true.
Proof.
  simpl.
```

```
Proving: negb_negb_true_eq_true

1 subgoal

-------------- (1/1)
true = true
```

The `simpl` tactic simplifies the goal by computation. In this case, `negb (negb true)` computes to `true`.

```
  reflexivity.
```

```
Proving: negb_negb_true_eq_true

No more subgoals
```

The `reflexivity` tactic proves goals where the two sides of an equality reduce to the same expression. It is stronger than a purely textual check: it can unfold definitions and compute enough to see that both sides are definitionally equal.

```
Qed.
```

If you are just getting started with theorem provers, focus here on the block that starts with `Proof` and ends with `Qed`. This is a Rocq proof script. It is written in the tactic language, while function definitions like `negb` and `andb` are written in the core term language, called Gallina.

In this simple example, `simpl` is not actually necessary, because `reflexivity` can perform the needed computation itself.

```
Theorem negb_negb_false_eq_false : negb (negb false) = false.
Proof.
  reflexivity.
Qed.
```

Here is the same proof written directly as a Gallina term. Note that we use `Definition` rather than `Theorem`.

```
Definition negb_negb_false_eq_false' :
    negb (negb false) = false :=
  eq_refl.
```

The term `eq_refl` is the canonical proof that an expression is equal to itself. Since both sides reduce to the same Boolean, `eq_refl` is accepted.

You can check that the theorem and the definition produce essentially the same proof object using the `Print` command.

```
Print negb_negb_false_eq_false.
Print negb_negb_false_eq_false'.
```

```
negb_negb_false_eq_false = eq_refl
     : negb (negb false) = false

negb_negb_false_eq_false' = eq_refl
     : negb (negb false) = false
```

If you try to prove something incorrect, the prover will let you know.

```
Theorem fail_to_unify : negb (negb false) = true.
Proof.
  Fail reflexivity.
Admitted.
```

```
The command has indeed failed with message:
Unable to unify "true" with "negb (negb false)".
```

The command `Fail reflexivity.` asks Rocq to check that `reflexivity` fails. This is useful for experimentation. The theorem is then closed with `Admitted`, which means that the statement is accepted without proof.

> **Warning**
>
> `Admitted` is not a real proof. It is useful while experimenting or developing a file, but admitted theorems should not be treated as established results in finished work.

Now let us prove the general theorem. We need to prove the statement for an arbitrary Boolean `b`. Since `b` can only be `true` or `false`, it is natural to split the proof into two cases.

The `destruct` tactic plays the same role in proofs that the `match` keyword plays in function definitions.

- In a function definition, `match b with ... end` defines a result by considering all possible forms of `b`.
- In a proof, `destruct b` proves the current goal by considering all possible forms of `b`.

Since `bool` has exactly two constructors, `destruct b` creates two subgoals: one for `b = true` and one for `b = false`.

```
Theorem negb_inv (b : bool) : negb (negb b) = b.
Proof.
  destruct b.
```

```
Proving: negb_inv

2 subgoals

-------------- (1/2)
negb (negb true) = true
-------------- (2/2)
negb (negb false) = false
```

After `destruct b`, the goal has been modified. In the first case, `b` has been replaced by `true`; in the second case, `b` has been replaced by `false`.

We use focusing dashes to separate the two subgoals.

```
  - simpl. reflexivity.
```

```
Proving: negb_inv

This subproof is complete, but there are some unfocused goals:

-------------- (1/1)
negb (negb false) = false
```

The second case can be solved by `reflexivity` alone.

```
  - reflexivity.
```

Instead of using `reflexivity`, we can also reuse a previous proof with the `exact` tactic. The theorem `negb_negb_false_eq_false` has exactly the type needed for the second goal.

```
  - exact negb_negb_false_eq_false.
Qed.
```

One may observe that `negb_inv true` is a proof of the equality

```
negb (negb true) = true
```

and that `negb_inv` itself is a proof of the universally quantified proposition

```
forall b : bool, negb (negb b) = b
```

```
Check negb_inv true.
Check negb_inv.
```

```
negb_inv true
     : negb (negb true) = true

negb_inv
     : forall b : bool, negb (negb b) = b
```

> **Note**
>
> A theorem with a variable, such as `Theorem negb_inv (b : bool) : ...`, behaves like a function that takes an argument `b : bool` and returns a proof of the corresponding statement.

Equivalent Definitions of Implication

Recall the formulas from classical logic:

```
(P   Q)   ¬P   Q
(P   Q)   ¬(P   ¬Q)
```

The first formula is often taken as a definition of implication in classical logic.

In this section, we interpret the logical connectives as Boolean functions:

Logical notation	Boolean function
`P   Q`	`implb b b'`
`¬P`	`negb b`
`P   Q`	`orb b b'`
`P   Q`	`andb b b'`

Since we are working with Boolean-valued functions, saying that two Boolean formulas are equivalent means saying that they compute to the same Boolean value for all inputs. Therefore, we express equivalence here using equality of Booleans.

Later, when working directly with propositions in `Prop`, logical equivalence will be expressed differently, for example using `<->`.

We first prove the Boolean version of

```
(P   Q)   ¬P   Q
```

This exercise also shows how to structure a proof with pattern matching on several variables. First, we present a nested version: we destruct `b`, and then destruct `b'` in each branch. This gives two cases corresponding to the possible values of `b`, with two subcases corresponding to the possible values of `b'`.

```
Theorem implb_eq_orb_negb (b b' : bool) :
    implb b b' = orb (negb b) b'.
Proof.
  destruct b.
  - destruct b'.
    + reflexivity.
    + reflexivity.
  - destruct b'.
    all: reflexivity.
Qed.
```

The tactic `all: reflexivity` applies `reflexivity` to all remaining subgoals. Here, after destructing `b'`, both remaining cases can be solved in the same way.

Next, we prove the Boolean version of

```
(P   Q)   ¬(P   ¬Q)
```

Here we destruct `b` and `b'` simultaneously. This immediately creates four cases, much like the simultaneous pattern matching used in the definition of `andb`.

```
Theorem implb_eq_negb_andb_negb (b b' : bool) :
    implb b b' = negb (andb b (negb b')).
Proof.
  destruct b, b'.
  all: reflexivity.
Qed.
```

Notice that we used explicit parameters `(b b' : bool)` rather than implicit parameters `{b b' : bool}`. Braces introduce implicit arguments, which Rocq tries to infer from context. This is useful later, but explicit parameters are clearer at this introductory stage.

```
End BooleanValues.
```

3.4 Summary

In this lecture, we saw the following ideas:

- `true` and `false` are the two canonical values of type `bool`.
- `bool` is classified by the universe `Set`.
- Boolean functions can be defined by pattern matching.
- Functions of several arguments are usually curried in Rocq.
- The wildcard pattern `_` means that the value is irrelevant in that branch.
- `if ... then ... else ...` is a convenient notation for pattern matching on Booleans.
- Theorems are types, and proofs are terms inhabiting those types.
- `simpl` computes in the goal, while `reflexivity` proves equalities whose two sides reduce to the same expression.
- `destruct` performs case distinction in proofs, analogous to `match` in programs.
- Boolean equivalence of Boolean-valued expressions can be expressed as equality of Booleans.

References

4 Resources

Books & other resources

4.1 SML

- Elements of Functional Programming. Reade
- Introduction to Programming Using SML. Hansel, Rischel
- Programming in SML. Harper
- Programmierung: Eine Einfuehrung in Standard ML. Gert Smolka

4.2 Haskell

- Functional Programming with Gofer. Fokker
- Introduction to Computation, Haskell, Logic, and Automata. Wadler
- Introduction to Functional Programming using Haskell. Bird
- Introduction to Functional Programming Systems Using Haskell
- Thinking Functionally with Haskell. Bird
- Programming in Haskell. Graham Hutton
- Haskell - The Craft of Functional Programming. Thompson

4.3 OCAML

- OCaml from the very beginning. Whitington
- More OCaml. Whitingnton
- OCaml - Functional Programming for the Masses. Madhavapeddy

4.4 Rocq / Coq

- Mathematical Components. Assia Mahboubi, Enrico Tassi.
- Software Foundations. Benjamin C. Pierce
- Programs and Proofs - Mechanizing Mathematics with Dependent Types. Ilya Sergey
- Certified Programming with Dependent Types. Adam Chipala
- Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions. Bertot, Casteran